Conventions de nommage – Control Expert

Document ressource

Cours 0.1

v2026-11-04

IUT d'Annecy, 9 rue de l'Arc en Ciel, 74940 Annecy

Conventions de nommage – Programmation Control Expert

Sommaire

1 Introduction **2**

2 Variables booléennes issues de l'IO-Scanning **2**

3 Types de données et champs de structure **3**

3.1 Deux familles de types . . . 3

3.2 Nommage des champs de structure . . . 3

3.3 Types de transformation géométrique et de mouvement (uniVALplc) . . . 4

4 Blocs fonctionnels **4**

4.1 Préfixes de bibliothèque . . . 5

4.2 Blocs `VAL_` – échanges avec la baie CS9 . . . 5

4.3 Blocs `MC_` – commandes de mouvement (PLCopen) . . . 5

4.4 Blocs `MC_` – commandes de contrôle de mouvement . . . 6

4.5 Convention de nommage des instances . . . 6

5 Sections de programme, GEMMA et SFC **6**

5.1 Sections liées aux échanges Ethernet/IP . . . 6

5.2 Sections du GEMMA (`GM_`) . . . 7

5.3 Sections d'un mode de fonctionnement . . . 7

5.4 Étapes et transitions d'un grafcet fonctionnel . . . 7

5.5 Macro-étapes SFC (extension Control Expert) . . . 8

6 Réseau et DTM **8**

6.1 Adressage IP de la cellule (rappel) . . . 8

7 Objets et instances métier **9**

8 Synthèse et incohérences signalées **9**

8.1 Récapitulatif des incohérences relevées entre documents source 9

8.2 Glossaire 9

1 Introduction

Ce document rassemble et structure les **conventions de nommage** rencontrées dans le développement d'une application **Control Expert** pilotant une cellule robotisée associant un automate **Schneider M340**, un robot **Stäubli Scara** via le serveur client **uniVALplc**, un contrôleur **M221** et un pupitre opérateur **HMI**.

Les conventions présentées ici sont **issues** de cinq documents pédagogiques source (`RIF_GT_TP1_2024`, `RIF_RBT_TP2_2024`, `RIF_RBT_TP3_2024`, `RIF_RBT_TP4_2024`, `RIF_RBT_TP5_2024`) rédigés par un autre enseignant pour une cellule similaire. Elles ne constituent pas des énoncés de TP : seules les conventions de nommage elles-mêmes (préfixes, types, blocs fonctionnels, sections) ont été retenues et réorganisées par catégorie, indépendamment du fil pédagogique des TP dont elles sont issues.

Remarque *Allègement de la convention*

La convention d'origine, inspirée de la **notation hongroise** (préfixe `m_` suivi d'une ou plusieurs lettres de type, préfixes `nix`/`nqx`, instances `Inst<Bloc><Mode>`), a été **allégée** : le type d'une variable n'est plus encodé dans son nom, l'environnement de développement l'affichant à la complétion et au survol. Ce choix s'aligne sur les pratiques actuelles (PLCopen, Codesys, TwinCAT) : des noms métier courts en camelCase. Les types (`T<Nom>`/`T_<Nom>`), les objets métier (`<rôle><Équipement>`) et les sections/transitions du GEMMA sont inchangés. La table de correspondance ancien/nouveau nom figure dans `MIGRATION_NOTATION.md`.

Chaque tableau précise, pour une catégorie donnée, la convention, sa signification et un exemple conforme à la convention allégée.

2 Variables booléennes issues de l'IO-Scanning

Les variables booléennes **autonomes** obtenues par IO-Scanning Ethernet/IP (non forçables, front non accessible directement), c'est-à-dire déclarées hors d'une structure dédiée d'entrées/sorties, portent le préfixe `in` (entrée) ou `out` (sortie). Les champs des structures dédiées `inputs`/`outputs` n'ont aucun préfixe : l'appartenance à la structure suffit à indiquer leur nature.

Préfixe	Signification	Exemple
`in<Nom>`	Entrée booléenne autonome produite par un équipement scruté.	`inAid`, `inDgt`, `inDpp`, `inDpo`
`out<Nom>`	Sortie booléenne autonome consommée par un équipement scruté.	`outBE`, `outBP`, `outCR`, `outCV`, `outCB`
`<nom>`	Entrée physique du pupitre HMI, champ de la structure `inputs` (cf. 3).	`j1Plus`, `auto`, `emptyTanks`, `stopRequest`
`<nom>`	Sortie physique du pupitre HMI, champ de la structure `outputs`.	`redColumn`, `armUnderPower`
`<nom>`	Tableau de bits, champ de structure.	`hex7Seg`

Remarque *Fin de l'ambiguïté `ix/nix`*

Les documents source utilisaient indifféremment `m_ixJ1Plus` et `m_nixJ1Minus` au sein de la même structure `THmiInputs`. La convention allégée supprime ces préfixes dans les champs de structure : `j1Plus`, `j1Minus`. L'incohérence disparaît.

3 Types de données et champs de structure

3.1 Deux familles de types

Les documents source distinguent nettement deux familles de types, à ne pas confondre.

Préfixe de type	Signification	Exemple
`T<Nom>`	Type maison, défini par le développeur de l'application (STRUCT ou enum).	`THmi`, `THmiInputs`, `THmiOutputs`, `TTank`, `TCtrlStatusGroup`, `TMoveAbsolute`, `TPupitre`
`T_<Nom>`	Type de la bibliothèque cliente `uniVALplc` (Stäubli), fourni par le constructeur.	`T_StaeubliRobot`, `T_Status`, `T_Command`, `T_CommandInterface`, `T_FromRobot`, `T_ToRobot`, `T_CartesianPos`, `T_JointPos`, `T_Trsf`

3.2 Nommage des champs de structure

Quel que soit le type de la structure, ses champs portent simplement leur **nom métier** en camelCase, avec une minuscule initiale, **sans** préfixe `m_` ni lettre de type.

Forme	Type de champ	Exemple
`<nom>`	`BOOL`	`enable`, `busy`, `done`, `error`, `execute`, `active`
`<nom>`	`UDINT`	`errorID`
`<nom>`	`TIME`	`elapsedTime`
`<nom>`	`INT`	`stateGmma`, `curvSpeed_mm_per_s`, `movementID`, `partNumber`
`<nom>`	`REAL`	`velocity`, `acceleration`, `cartesianVel`
`<nom>`	`UINT`	`coordSystem`, `toolNumber`, `bufferMode`
`<nom>`	`ARRAY`	`hex7Seg`, `tankLine` (`ARRAY OF TTank`)
`<nom>`	Type composé	`cartesianPos` (`T_CartesianPos`), `jointPos` (`T_JointPos`), `transitionParam` (`MC_transitionParameter`), `inputs/outputs` (`THmiInputs/THmiOutputs`)

CC BY NC SA

Remarque *Fin de l'ambiguïté `m_int/m_i`*

Les documents source utilisaient deux préfixes pour un champ `INT` (`m_intPartNumber` au TP2, `m_iStateGmma` au TP3). La lettre de type étant supprimée, ces champs deviennent `partNumber` et `stateGmma`.

3.3 Types de transformation géométrique et de mouvement (uniVALplc)

Type	Rôle	Champs / remarque
`T_CartesianPos`	Position cartésienne absolue (repère `world` ou repère utilisateur).	Utilisé par `MC_MoveLinearAbsolute`, `MC_MoveDirectAbsolute`, `VAL_ShiftPoint`.
`T_JointPos`	Position articulaire absolue (un champ `J1`..`J6` par axe, `REAL`).	Utilisé par `MC_MoveAxisAbsolute`. Pour un robot 4 axes, `J5` et `J6` valent 0.
`T_Trsf`	Transformation géométrique relative (offset `X, Y, Z, RX, RY, RZ`, tous `REAL`).	Utilisé avec `VAL_ShiftPoint` pour un calcul de position relatif.
`eMC_BUFFER_MODE`	Enum du mode d'empilement d'une commande de mouvement (`Aborting:=0`, `Buffered:=1`, `BlendingJoint:=6`, `BlendingCartesian:=7`).	Champ `BufferMode`/`bufferMode` des blocs de mouvement.
`eMC_TRANSITION_MODE`	Enum du profil de transition entre deux mouvements enchaînés (`None:=0`, `Corner distance:=3`, `Blending distances:=10`).	Champ `TransitionMode`/`transitionMode`.
`MC_TransitionParameter`	Structure `leave`/`reach` (distances de sortie/reprise de trajectoire, `REAL`).	Champ `TransitionParam`/`transitionParam`.
`TMoveAbsolute` / `TMoveAxisAbs`	Type maison regroupant tous les paramètres d'appel d'un bloc `MC_Move...Absolute` (entrées, sorties, position) pour en faciliter le passage.	`TMoveAbsolute` pour les mouvements cartésiens (TP4), `TMoveAxisAbs` pour les mouvements articulaires (TP5) – structure quasi identique, seul le nom diffère selon le TP source.

4 Blocs fonctionnels

4.1 Préfixes de bibliothèque

Préfixe	Origine / rôle	Exemple
`VAL_`	Bloc fonctionnel de la bibliothèque cliente `uniVALplc` (Stäubli), non spécifique au standard PLCopen.	`VAL_ReadAxesGroup`, `VAL_WriteAxesGroup`, `VAL_ReadWriteIO`, `VAL_ShiftPoint`
`MC_`	Bloc fonctionnel conforme au standard **PLCopen** pour la commande de mouvement (*Motion Control*).	`MC_MoveLinearAbsolute`, `MC_MoveDirectAbsolute`, `MC_MoveAxisAbsolute`, `MC_MoveCircularAbsolute`, `MC_GroupPower`, `MC_GroupReset`, `MC_GroupStop`, `MC_GroupInterrupt`, `MC_GroupContinue`
`FB_`	Bloc fonctionnel maison de l'application.	`FB_ReadHmi`, `FB_WriteHmi`

4.2 Blocs `VAL_` – échanges avec la baie CS9

Bloc	Rôle	Type associé
`VAL_ReadAxesGroup`	Lit l'état du robot (baie → automate) en début de cycle.	`T_FromRobot` (dans `T_StaeubliRobot`)
`VAL_WriteAxesGroup`	Envoie les commandes de mouvement établies durant le cycle (automate → baie) en fin de cycle.	`T_ToRobot`
`VAL_ReadWriteIO`	Accès direct aux E/S physiques du contrôleur CS9 (connecteur J212).	–
`VAL_ShiftPoint`	Calcule une position cartésienne par application d'une transformation géométrique relative (équivalent `compose()`/`appro()`).	`T_Trsf`, `T_CartesianPos`

4.3 Blocs `MC_` – commandes de mouvement (PLCopen)

Bloc	Type de mouvement	Position finale
`MC_MoveAxisAbsolute`	Interpolation articulaire, sans modélisation d'outil (uniquement pour un Scara).	`T_JointPos`
`MC_MoveDirectAbsolute`	Interpolation articulaire.	`T_CartesianPos`
`MC_MoveLinearAbsolute`	Mouvement linéaire.	`T_CartesianPos`
`MC_MoveCircularAbsolute`	Mouvement circulaire.	`T_CartesianPos`

4.4 Blocs MC_ – commandes de contrôle de mouvement

Bloc	Rôle
MC_GroupPower	Met le bras sous/hors puissance (entrée Enable).
MC_GroupReset	Acquitte les défauts pendants côté robot.
MC_GroupStop	Arrêt contrôlé du mouvement en cours et vidage complet de la pile de commandes empilées.
MC_GroupInterrupt	Arrête le mouvement en cours *sans* vider la pile (reprise possible via MC_GroupContinue).
MC_GroupContinue	Reprend un mouvement arrêté par MC_GroupInterrupt, ou autorise à nouveau l'exécution de la pile après un MC_GroupReset/MC_GroupStop.

4.5 Convention de nommage des instances

Le nom d'une instance de bloc fonctionnel ne reprend ni préfixe Inst ni nom technique du bloc : il se compose de son **rôle** (fonction résumée en un mot) suivi du mode qui l'utilise. Par exemple une instance de MC_GroupReset réservée pour le mode D2 sera nommée resetD2. Une instance commune à tous les modes n'a pas de suffixe : readAxes, writeAxes, power. Les objets passés en paramètre (de type TCtrlStatusGroup/TCtrlStatusGroupPower) sont préfixés par status et reprennent le rôle et le mode : statusResetD2, statusStopD2, statusContinueD2 (TP3), ou statusStopF4, statusContinueF4 (TP5).

Convention	Signification	Exemple
<rôle><Mode>	Instance d'un bloc fonctionnel, suffixée par le mode qui l'utilise (sans suffixe si commune à tous les modes).	powerD1, resetD2, stopD2, continueD2, readAxes
status<Rôle><Mode>	Objet TCtrlStatusGroup de passage de paramètres, suffixé par le rôle et le mode.	statusStopF4, statusContinueF4, statusPower
moveAxisLinearAbs<Mode> / moveAxisAbsolute<Mode>	Donnée de type TMoveAbsolute/TMoveAxisAbs facilitant l'appel de l'instance, suffixée par le mode.	moveAxisLinearAbsF1 (TP4), moveAxisAbsoluteF4 (TP5)
trsfPos<Mode>	Objet T_Trsf pour le calcul en relatif, suffixé par le mode.	trsfPosF1

5 Sections de programme, GEMMA et SFC

5.1 Sections liées aux échanges Ethernet/IP

Section	Rôle	Ordre de scrutation
ReadEthIpRobot	Lit les échanges Ethernet/IP avec la baie robot (VAL_ReadAxesGroup).	En premier dans le cycle API.
WriteEthIpRobot	Écrit les échanges Ethernet/IP vers la baie robot (VAL_WriteAxesGroup).	En dernier dans le cycle API.

5.2 Sections du GEMMA (GM_)

Les trois sections du GEMMA sont scrutées dans un ordre fixe, juste après ReadEthIpRobot : GM_GmmaPre, puis GM_Gmma, puis GM_GmmaPost.

Section	Rôle	Langage
GM_GmmaPre	Initialise le graphe (1er cycle en RUN ou erreur de communication détectée via les bits de santé) ; positionne la période du bit de vie (Command.LifebitPeriod).	ST
GM_Gmma	Traduction du GEMMA en langage SFC (une étape par mode : GM_S_D1, GM_S_D2, GM_S_A5, GM_S_A6, GM_S_A1, GM_S_F1, GM_S_A2, GM_S_F4...).	SFC
GM_GmmaPost	Positionne les bits de scrutation conditionnelle de chaque mode (xD1, xD2, xA5, xA6, xA1, xF1A2, xF4...) et fournit l'état hexadécimal du GEMMA à l'écran HMI.	ST

Convention	Signification	Exemple
GM_S_<Mode>	Étape SFC du GEMMA associée au mode <Mode>.	GM_S_D1, GM_S_F1
GM_T_<ModeA>to<ModeB>	Transition SFC entre deux modes du GEMMA.	GM_T_D1toD2, GM_T_A1toF1, GM_T_F1toA2
GM_AP1_InitChart<Mode>	Action de type P1 associée à une étape du GEMMA, initialisant le graphe SFC du mode (utile pour les modes non répétitifs).	GM_AP1_InitChartD2, GM_AP1_InitChartA5, GM_AP1_InitChartA6
x<Mode>	Bit booléen autorisant la scrutation conditionnelle des sections d'un mode.	xD1, xD2, xA5, xA6, xA1, xF1A2 (mode F1 *et* A2), xF4

5.3 Sections d'un mode de fonctionnement

Section	Rôle	Langage
<Mode>_<NomDuMode>	Traduction SFC du grafcet fonctionnel du mode (structure de Mealy).	SFC
<Mode>_<NomDuMode>Post	Synthèse des sorties/actionneurs, scrutation des instances de blocs fonctionnels, en fonction des bits d'activité des étapes. Postérieure au graphe SFC.	ST

Exemples rencontrés : D1_ArretUrgence (ST, section unique – pas de graphe nécessaire), D2_Diagnostic/D2_DiagnosticP
A5_RemiseEnRoute/A5_RemiseEnRoutePost, A6_MiseEnCI/A6_MiseEnCIPost, A1_ArretEnCi (ST seul), F1_ProductionNorm
F4_ModeManuel/F4_ModeManuelPost.

5.4 Étapes et transitions d'un grafcet fonctionnel

Convention	Signification	Exemple
XX_S_i	Étape i du grafcet fonctionnel du mode XX.	F1_S_0, F1_S_12
XX_T_<...>	Transition nommée du grafcet fonctionnel du mode XX.	F1_T_MiseEnService

5.5 Macro-étapes SFC (extension Control Expert)

Règles de nommage spécifiques aux macro-étapes et à leur expansion, introduites au TP4.

Convention	Signification	Exemple
`XX_MSI`	Macro-étape : `XX` = mode de fonctionnement, `MS` = Macro-Step, `I` = numéro de la macro-étape dans le mode.	`F1_MS1`, `F1_MS2`
`XX_MSI_J`	Étape `J` de l'expansion de la macro-étape `XX_MSI`.	`F1_MS0_2`
`XX_MSI_T_`	Préfixe d'une section de transition de la macro-étape `XX_MSI`.	`F1_MS0_T_MiseEnService`
`XX_MSI_IN` / `XX_MSI_OUT`	Étape d'entrée / de sortie de la macro-étape `XX_MSI` (nom imposé).	`F1_MS1_IN`, `F1_MS1_OUT`

6 Réseau et DTM

Convention	Signification	Exemple
`NOC_ETHIP_<CIBLE>`	Nom du DTM d'un coupleur réseau Ethernet/IP, suffixé par la cible scrutée.	`NOC_ETHIP_CONVOYEUR`, `NOC_ETHIP_ROBOT`
`<nom métier court>`	Nom d'un équipement ajouté à la configuration réseau : nom métier court, sans préfixe.	`guichet`, `cs9Scara`, `m221Scara`

6.1 Adressage IP de la cellule (rappel)

Équipement	Réseau	Adresse
PLC M340	Privatif salle (classe B)	172.16.180.180 (primaire)
PLC M340	Ethernet/IP	192.168.0.1 (secondaire)
PFC200 (Guichet)	Privatif salle	172.16.180.190
CS9 (robot)	Privatif salle / Ethernet/IP	172.16.180.30 (J205) / 192.168.0.10 (J207-J208)
M221 (E/S)	Ethernet/IP	192.168.0.11
HMI	Ethernet/IP	192.168.0.12
Stations PC	Privatif salle	172.16.180.i, $i \in [1, 17]$

7 Objets et instances métier

Convention générale : `<rôle><Équipement>` en camelCase, où `<rôle>` précise la nature de l'échange (`from`/`to` pour un flux avec un équipement scruté) et `<Équipement>` est un nom métier ou un suffixe identifiant l'équipement/la station concernée (ex. `Scara`).

Objet	Rôle	Type
`staeubliRobotScara`	Représente le robot et son contrôleur CS9.	`T_StaeubliRobot`
`fromRobotScara`	Données produites par le robot, lues en début de cycle.	`T_FromRobot`
`toRobotScara`	Commandes consommées par le robot, écrites en fin de cycle.	`T_ToRobot`
`fromHmiScara` / `toHmiScara`	Données lues/écrites depuis/vers le pupitre HMI.	`TfromHmi` / `TtoHmi`
`statusRead` / `statusWrite`	Objets d'état de contrôle associés à la lecture/écriture du groupe d'axes.	`TCtrlStatusGroup`
`hmiScara`	Instance représentant l'état complet du pupitre HMI pour la station Scara.	`THmi`

Remarque *Accès à un champ imbriqué*

Les objets de type `THmi` s'utilisent avec la notation pointée classique de l'IEC 61131-3, par exemple `hmiScara.outputs.scree` pour écrire l'état du GEMMA en hexadécimal à l'écran, ou `hmiScara.outputs.toR.armUnderPower` pour piloter un voyant de synthèse.

8 Synthèse et incohérences signalées

8.1 Récapitulatif des incohérences relevées entre documents source

Point ambigu	Description
`m_int` vs `m_i`	Deux préfixes pour un champ `INT` dans les documents source. **Résolu** par la convention allégée (plus de lettre de type) – cf. 3.
`m_ix` vs `m_nix`	Préfixes mélangés dans `THmiInputs` (TP5). **Résolu** : les champs de structure n'ont plus de préfixe – cf. 2.
`TMoveAbsolute` vs `TMoveAxisAbs`	Deux noms de type distincts pour une structure de paramétrage de mouvement quasi identique, selon que le TP source traite un mouvement cartésien (TP4) ou articulaire (TP5) – cf. 3.

8.2 Glossaire

GEMMA *Guide d'Étude des Modes de Marches et d'Arrêts* – outil de description des modes de fonctionnement et d'arrêt d'un système automatisé (états `D1`, `D2`, `A1`, `A5`, `A6`, `F1`, `F4`...).

Macro-étape Extension du langage SFC (Control Expert) permettant de regrouper une expansion d'étapes sous une étape unique, afin de limiter le niveau de détail visible dans un graphe.

Multi-jetons Extension du langage SFC (Control Expert) autorisant plusieurs étapes initiales dans un même graphe.

Machine de Mealy Structure d'implantation d'un automatisme où les sorties dépendent à la fois de l'état courant et des entrées (utilisée ici pour la partie postérieure au graphe SFC, en langage ST).

TCP *Tool Center Point* – point de référence de l'outil du robot, dont la trajectoire et la vitesse sont pilotées par les commandes de mouvement.

Bit de vie Bit rafraîchi périodiquement (champ `Command.LifebitPeriod`) permettant à la baie CS9 de détecter une perte de communication avec l'automate.

Sas (guichet) Principe de gestion de flux de palettes à capacité unitaire : une palette ne peut entrer dans le poste que si celui-ci est libre.